Assignment 2

Aim: To Implement an AVL tree in C, perform sequential Insertions of the months (December, January, April, March, July, August, October, February, November, May, June) using lexicographical comparison.

Algorithm:

1. Define Node structure containing a string key and pointers left and right child nodes.

2. Write helper function

- Define height(node) to return the height of a given node.
- Define getbalance(node) as height(left) - height(right)
- Define rightRotate(y) and leftRotate(x) to restructure unbalanced nodes and update their heights.

3. Insertion

(i) If the tree or subtree is empty, allocate and return a new node with height 1

(ii) Compare string with the current nodekey using strcmp();

- If smaller, recursively call insert on left subtree.
- If larger recursively call insert on right subtree.

(iii) Update the height of ancestor node
height = 1 + max(height(left), height(right))

(iv) Calculate the Balance factor of the current node.

(v) If the balance factor is unbalanced (>1 or <-1) appl Rotation

- LL case (balance > 1 and key < left → key)
  call rightRotate(node),

- RR case (balance < -1 and key > right → key)
  call leftRotate(node)

- LR case (balance > 1 and key < left → key)
  call leftRotate(node->left) then rightRotate(node)

- RL case (balance < -1 and key < right → key): call rightRotate(node->r
  the leftRotate(node);

4. Display
- print the Tree via preorder traversal.
- print the tree structure visually using reverse in order traversal with indentation.

Conclusion:- The AVL Tree algorithm was successfully implemented in C. and dynamically balance a set of 11 months names.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char key[20];
    struct Node *left;
    struct Node *right;
    int height;
};

int max(int a, int b) {
    return (a > b) ? a : b;
}

int height(struct Node *n) {
    if (n == NULL)
        return 0;
    return n->height;
}

struct Node* createNode(const char *key) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    strcpy(node->key, key);
    node->left = NULL;
    node->right = NULL;
    node->height = 1;
    return node;
}

struct Node* rightRotate(struct Node *y) {
    struct Node *x = y->left;
    struct Node *T2 = x->right;
```

```
    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

struct Node* leftRotate(struct Node *x) {
    struct Node *y = x->right;
    struct Node *T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

int getBalance(struct Node *n) {
    if (n == NULL)
        return 0;
    return height(n->left) - height(n->right);
}

struct Node* insert(struct Node* node, const char *key) {
    if (node == NULL)
        return createNode(key);
```

```
    int cmp = strcmp(key, node->key);

    if (cmp < 0)
        node->left = insert(node->left, key);
    else if (cmp > 0)
        node->right = insert(node->right, key);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right));

    int balance = getBalance(node);

    if (balance > 1 && strcmp(key, node->left->key) < 0)
        return rightRotate(node);

    if (balance < -1 && strcmp(key, node->right->key) > 0)
        return leftRotate(node);

    if (balance > 1 && strcmp(key, node->left->key) > 0) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    if (balance < -1 && strcmp(key, node->right->key) < 0) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

void printTree(struct Node *root, int space) {
```

```
    if (root == NULL)
        return;

    space += 10;

    printTree(root->right, space);

    printf("\n");
    for (int i = 10; i < space; i++)
        printf(" ");
    printf("%s\n", root->key);

    printTree(root->left, space);
}

void preOrder(struct Node *root) {
    if (root != NULL) {
        printf("%s ", root->key);
        preOrder(root->left);
        preOrder(root->right);
    }
}

int main() {
    struct Node *root = NULL;
    const char *months[] = {
        "December", "January", "April", "March", "July",
        "August", "October", "February", "November", "May", "June"
    };

    int n = sizeof(months) / sizeof(months[0]);

    for (int i = 0; i < n; i++) {
```

```
        root = insert(root, months[i]);
    }

    printf("Pre-order traversal:\n");
    preOrder(root);
    printf("\n\n2D Tree Structure (sideways):\n");
    printTree(root, 0);

    return 0;
}
```